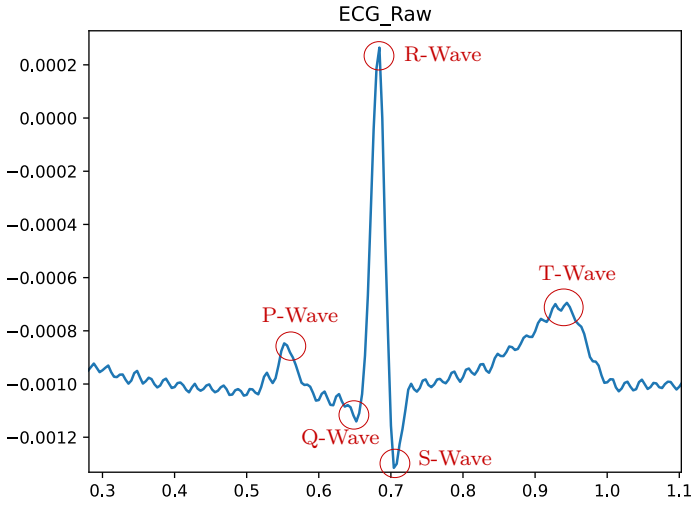
Digital Signal Processing
ECG FIR Filtering Assignment

2785209M & 2664680S

1 Plotting the ECG Signal

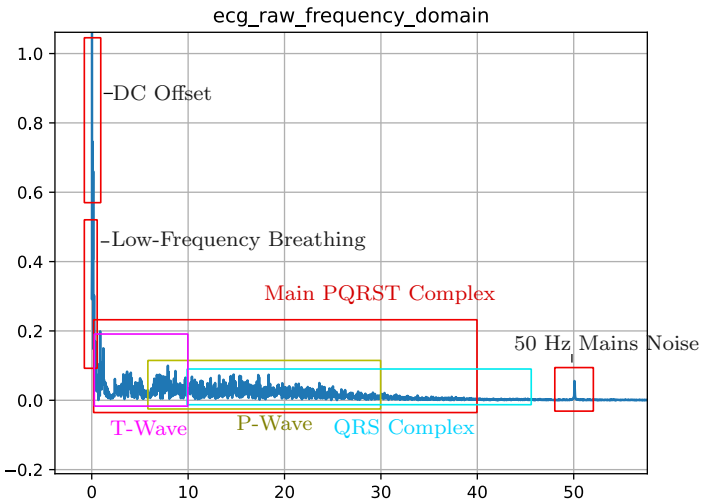
The experiment began by taking an ECG recording and plotting it in python using matplotlib. The electrodes were attached just above the hip and on the front of the shoulder.

1.1 Structure of PQRST Complex



It is important that we preserve the frequencies used to construct the PQRST complex, otherwise we would be destroying important data from the ECG.

1.2 Sources of Noise in the ECG



The main sources of noise in an ECG are 50Hz mains interference, Breathing, and random noise caused by movement. There is also a DC offset which shifts the entire signal by a certain amount, caused by potential difference formed between the electrodes and the skin.
The human heart rate can go as low as 0.5Hz at complete rest. Considering this, the filter should remove frequencies below 0.5Hz (This includes DC shift and Breathing) and above 45Hz (Mains noise and other high frequency interference). Although this leaves in random noise cause by movement, the core frequencies of the PQRST complex must be retained for the signal to remain accurate for monitoring purposes.

2 Generating the FIR Coefficients

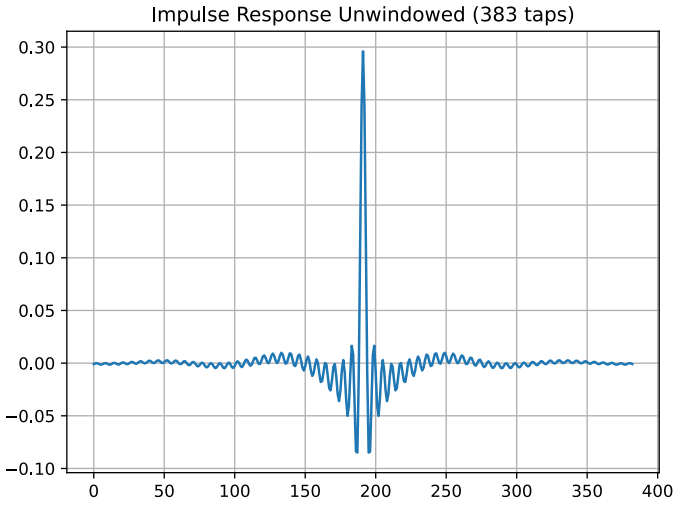
2.1 Stop-Band Filter

A stop-band filter was implemented in this code by utilizing an all-pass filter, a band-pass filter, and the principle of superposition. This was done instead of using a pure band-stop filter in order to keep the code simple and elegant, as it is easier to remove frequencies from an all-pass filter than it is to combine multiple separate lo-pass and hi-pass filters together.
First, the number of coefficients needed for the windowing function to work had to be calculated. For a Kaiser Window, this changes based on the desired stopband attenuation and transition width. The formula is displayed in the next subsection in Equation (3). Then, using the number of taps, an ideal all-pass filter was defined by creating an array of zeros, with an index stretching from indices of -m to m (where m = num_taps / 2), with the central value at index m set to 1. This represents the ideal impulse response. Finally the array n is initialized, which represents discrete time indices from -m to m. Evalutating functions over this symmetric time vector is what gives the filter linear phase and constant group delay.
This all-pass filter is then modified in a for loop iterating over the array of stop bands. The stop bands live in the frequency domain, so an inverse discrete time fourier transform is applied to convert them into the time-domain before they are subtracted from the impulse response:

$$h_{\text{band}}[n] = 2 \cdot f_2 \cdot \text{sinc}(2 \cdot f_2 \cdot n) - 2 \cdot f_1 \cdot \text{sinc}(2 \cdot f_1 \cdot n) \quad (1)$$

(Note: Doing the IDFT manually is less efficient than using np.IFFT() and was done purely for analytical purposes)

After this for-loop is complete, h will be a raw band-stop filter (sharp edges in the frequency domain, causing ringing due to the Gibbs Phenomenon). The Gibbs phenonenom can be seen in the impulse response of the filter as the side lobes aren't able to settle to zero before they get truncated:



To fix this, it must be multiplied with a windowing function.

2.2 The Kaiser Window

The Kaiser window was chosen for its flexibility and simplicity. It gives greater flexibility than other windowing functions, since the user has control over both the size of the transition window and the stopband attenuation. This allows you to find a precise balance between a narrow central peak (sharper cutoff) and flatter sidelobes (better noise rejection) in the impulse response. With specific stopband attenuations, it is also possible for the Kaiser window to mimic other windowing functions. Furthermore, the Kaiser filter has straightforward formulas to calculate beta (the stopband attenuation parameter) and num_taps (the number of coefficients needed for the filter), allowing it to be

practically calculated almost entirely in code without external libraries. The formulae used in the code are as follows:

beta parameter:

$$\beta = \begin{cases} 0.1102(A-8.7) & \text{if } A > 50 \\ 0.5842(A-21)^{0.4} + 0.07886(A-21) & \text{if } 21 \leq A \leq 50 \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

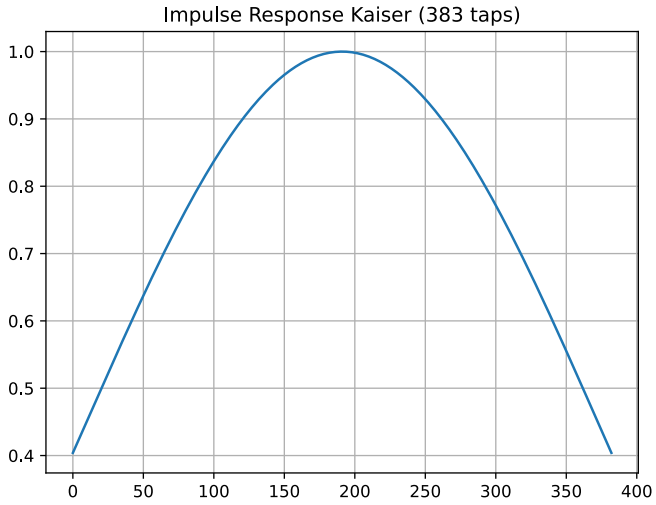
number of taps:

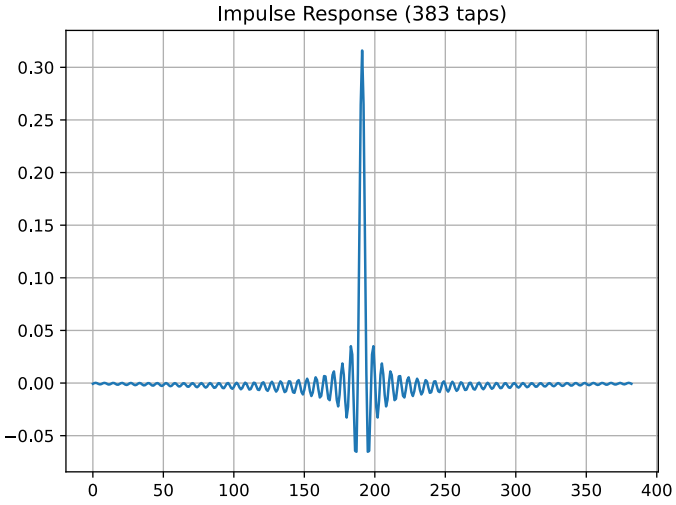
$$N = \left\lfloor \frac{A-8}{2.285 \cdot 2\pi \left(\frac{\Delta f}{f_s}\right)} \right\rfloor \quad (3)$$

kaiser window:

$$w[n] = \frac{I_0\left(\beta\sqrt{1-\left(\frac{n}{m}\right)^2}\right)}{I_0(\beta)} \quad (4)$$

The FIR filter is then multiplied by the Kaiser window, to create the final array of coefficients, producing this impulse response in the time domain:





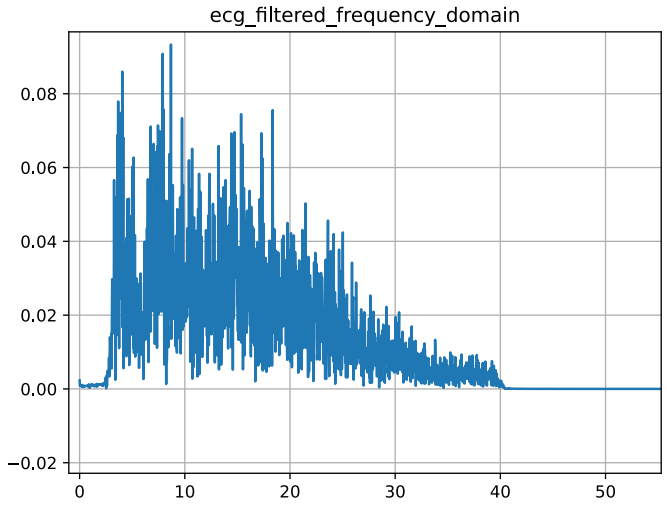
As you can see, this impulse response has a large, narrow central peak and sidelobes that gradually decay to zero as they approach tap 0 and tap 383. This shows that our windowing function has successfully removed ringing due to the Gibbs Phenomenon. These coefficients can now be passed into the fir_filter class to simulate realtime filtering.

3 Simulation of Real-time Filtering

In order to simulate realtime filtering of the data, a python class was created to represent the FIR class. This class takes the taps (calculated from the filter design function) as it's sole argument. Additionally, the class has 3 data members; coefficients is an array of the filter's taps; num_taps is the length of coefficients; And delay_line is a double ended queue with a maximum length of num_taps. A double ended queue was used as the data structure for the delay line because, when given a maximum length, it acts similarly to a ring buffer where adding a new value to a full queue causes one value to fall off the other end. This is an efficient implementation, because it does not require manual shifting of the delay line.
The function member used for simulating realtime processing is simply dofilter. This takes a scalar input of the newest sample in the data, appends it to the delay line and then loops through the delay line to calculate a cumulative sum of delay_line[i] * coefficients[i]. It returns the result of this summation. To simulate realtime input to the filter in python, a for loop iterates through the ecg data and passes it sample by sample to the FIR filter for processing.

3.1 Filter Design

$$MathBlock \quad (5)$$



tiny text Normal Sized text large text **bold text**

4 Adaptive LMS Filter

The doFilterAdaptive function member extends the FIR filter class for adaptive filtering. The arguments to this function are signal, noise, and learningRate. The signal argument is the signal that we are trying to remove from our ecg. The noise argument is our noisy ecg signal, and learningRate is the learning rate/step size of our adaptive algorithm. The function initially calls the doFilter function on the signal for the forward pass before updating the weights of the filter using a least mean squares method:

$$error = noise - filtered_signal \quad (6)$$

$$coefficients[i] = coefficients[i] + learningRate \cdot error \cdot delay_line[i] \quad (7)$$

Our signal input is a 50Hz sin wave plus a DC shift of 0.25. The value of the DC shift was found experimentally with the goal of removing the DC shift from the signal without degrading the quality of the PQRST wave. The learning rate was also found experimentally such that the filter coefficients converge quickly without exploding.

4.1 Comparison with FIR Filter

5 Heartbeat Detection

Heartbeat was detected using a matched filter. This process requires a heartbeat template which can then be correlated with the ECG signal to find the location of the heartbeats. The other team member's ECG signal was used to create this template. The ECG signal was first filtered using the FIR filter to remove low frequency drift and mains noise. An ideal PQRST wave was then extracted and reversed to use as the filter coefficients of the matched filter. The PQRST template has to be reversed in order for the convolution of the FIR filter to act as a correlation operation between the template and the ECG signal. The output for from the matched filter is then squared to amplify the

SNR. A threshold is set to detect the peaks of the output signal which correspond to heartbeats, and the heartrate is calculated as the inverse of the time between detected peaks. A heuristic is applied by setting a cooldown time between detected peaks which limits the heartrate to 220bpm. This helps to avoid any false positives.

5.1 Subsection